MCU Selection & Architecture

Choosing the right brain for your embedded product

8-bit vs 32-bit • Cortex-M • Peripherals • Toolchain

Audience: Beginner → Intermediate Engineering Students

Architecture • Cores • Memory • Peripherals • Power • Cost

Why MCU Choice Matters

The decision drives BOM, firmware, time-to-market

Performance ceiling

Decides what your firmware
can actually do.

Power budget

Sleep currents in µA
determine battery life.

Peripheral fit

USB / CAN / ADC / DMA —
missing one = redesign.

Toolchain & ecosystem

Free IDE, debugger,
community support.

Cost @ volume

Unit cost varies 10×
between MCU families.

Lifecycle

Supply chain — will you
buy this part in 5 years?

8-bit vs 16-bit vs 32-bit

Architecture comparison

Class	Example MCU	Clock	Use case
8-bit	ATmega328P, PIC16	≤ 20 MHz	Simple I/O, blinky, learning
8-bit (modern)	ATtiny / STM8	≤ 24 MHz	Cost-down embedded
16-bit	MSP430, PIC24	≤ 40 MHz	Low-power sensor nodes
32-bit (Cortex-M0)	STM32G030, RP2040	48-133 MHz	General embedded
32-bit (M4F)	STM32F4, nRF52, ESP32	≤ 240 MHz	IoT, wireless, motor control
32-bit (M7)	STM32H7, NXP i.MX RT	≤ 600 MHz	AI, real-time, audio
Linux-class	RPi RP1, BCM2711	≥ 1 GHz	Computer-like products

Default 2026

For most new projects, start at Cortex-M0/M0+ — they're cheaper than 8-bit at moderate volume and far more capable.

ARM Cortex-M Family

The dominant 32-bit MCU architecture

Cortex-M0

Cheapest, smallest.
No DSP, no FPU.
IoT, sensors.

Cortex-M0+

Slightly more efficient.
RP2040, STM32G0.
Best cost/perf.

Cortex-M3

Mid-tier, fast IRQ.
Legacy STM32F1.
Obsolete for new design.

Cortex-M4 / M4F

Add DSP + FPU.
Most popular core.
STM32F4, nRF52.

Cortex-M7

High-perf, dual-issue.
STM32H7, i.MX RT.
Audio, AI.

Cortex-M33

Add TrustZone-M.
Secure boot, IoT.
STM32U5, nRF53/54.

Memory — Flash, RAM, EEPROM

How much you need, where it lives

- Flash — code + constants. 32 KB = small, 512 KB = comfortable, 2 MB = serious
- RAM — stack, heap, buffers. 8 KB = tight, 64 KB = average, 256 KB+ = generous
- EEPROM (legacy) — small non-volatile storage. Many modern MCUs emulate it in Flash
- QSPI / OctoSPI — external flash up to 256 MB for graphics, ML models
- Cache / TCM — Cortex-M7 has internal SRAM banks accessible in 1 cycle
- Rule: target ≤ 70 % flash, ≤ 60 % RAM in production — leave headroom for features

Peripheral Decision Matrix

Make sure your MCU has what you need

- UART × 2-4 — debug + module comms
- SPI × 1-2 — sensors, SD card, display
- I²C × 1-2 — sensors, EEPROM
- USB FS/HS — host or device
- CAN / CAN-FD — automotive, industrial
- Ethernet MAC — Cortex-M7 typically

- ADC: ≥ 12-bit, ≥ 1 MSPS, multi-channel
- DAC: 12-bit if you need audio out
- Timers: PWM channels = motor channels × 2
- DMA: must-have for SPI/UART/ADC streaming
- RTC: battery-backed real-time clock
- Crypto + TRNG: secure-boot products

Power & Low-Power Modes

Battery-powered? Sleep current is everything

Mode	Typical I	Wake source	Use
Run @ 48 MHz	5-20 mA	Always	Active processing
Sleep	1-3 mA	Any IRQ	Pausing
Stop	5-50 µA	EXTI, RTC	Light sleep
Standby	1-5 µA	WKUP pin, RTC	Battery low
Shutdown	100-500 nA	RST, wake pin	Long storage

Calculation

Coin cell CR2032 = 220 mAh. At 5 µA average, life ≈ 220 / 0.005 ≈ 44000 h ≈ 5 years. At 5 mA, only ~44 hours. Sleep mode is the entire game for IoT.

Major MCU Vendors

Who you'll be buying from

ST (STM32)

Largest Cortex-M offering.
Great docs, free IDE.

Nordic

Best BLE / wireless.
nRF52, nRF53, nRF54.

Espressif

ESP32 family — Wi-Fi + BLE
at best price/performance.

NXP

i.MX RT high-end,
LPC mid-range.

Microchip

AVR / PIC / SAM —
legacy + Cortex-M.

Raspberry Pi

RP2040 / RP2350 —
best community, lowest cost.

Toolchain & Debugger

What you'll actually develop with

- IDE — vendor-supplied (STM32CubeIDE, MCUXpresso) OR VS Code + extensions
- Compiler — arm-none-eabi-gcc (free), IAR (commercial), Keil MDK (commercial)
- Build system — CMake, Make, PlatformIO, Zephyr Build, Mbed
- RTOS — FreeRTOS (free), Zephyr (open-source), ThreadX (Microsoft)
- Debugger probe — ST-Link, J-Link, DAPLink — JTAG / SWD interface
- RTT / SWO trace — live debug output without UART
- SDK — vendor HAL + middleware (USB, Bluetooth, TCP/IP)

Selection Checklist

Twelve questions before you choose

- What's the worst-case CPU load?
- How much flash & RAM after libraries?
- Which peripherals required (exact count)?
- Battery-powered? Sleep current target?
- Wireless needs (BLE / Wi-Fi / LoRa)?
- Real-time deadlines (audio, motor)?
- Volume — unit cost at 10k pcs?
- Lifecycle — guaranteed availability?
- Toolchain availability and licenses?
- Existing firmware to port?
- Security needs (secure boot, crypto)?
- Certification (automotive / medical)?

Recommended Dev Boards

Start here — don't design custom on day 1

STM32 Nucleo

₹1500-3000 — F4, L4, U5.
Full-featured, Arduino headers.

Raspberry Pi Pico

₹400 — RP2040.
Lowest cost, MicroPython.

ESP32-DevKitC

₹500 — Wi-Fi + BLE.
For IoT prototyping.

nRF52 DK

₹3000 — BLE expert kit.
Nordic SDK.

Arduino Uno R4

₹2000 — Renesas RA4M1.
Massive community.

Teensy 4.x

₹3000 — i.MX RT @ 600 MHz.
Audio, USB power.

Common MCU Selection Mistakes

What experience teaches

- Picking by clock speed alone (irrelevant for I/O-bound apps)
- Under-sizing flash — running out of space mid-project
- Missing peripheral count (need 3 SPI, picked 2)
- Wrong package (LQFP vs QFN) — affects assembly cost
- Ignoring sleep current — battery life misses spec
- No second source — vendor goes EOL during production
- Ignoring crypto / secure-boot — fails compliance
- Choosing exotic MCU — toolchain & community thin

AI / ML on MCUs

TinyML — the new frontier

- Cortex-M55 + Ethos-U55 — dedicated NPU for inference
- STM32U5/H7 with CMSIS-NN library — efficient quantised models
- TensorFlow Lite for Microcontrollers — popular runtime
- Use cases: keyword spotting, gesture, anomaly detection, predictive maintenance
- Memory budget: model + tensor arena < free RAM
- Vendors with ML-ready MCUs: ST (STM32Cube.AI), NXP (eIQ), Renesas (RAFAst)
- Lots of open-source examples at Edge Impulse, TFLite Micro repo

Best Practices Recap

Habits of a senior firmware engineer

- Define performance & memory budgets first
- Pick a vendor with strong toolchain & docs
- Choose a part with 50% headroom in flash / RAM
- Prefer LQFP/QFN with exposed pad for thermal
- Specify package, temp grade, lifecycle status

- Second source from same vendor (pin-compatible variant)
- Buy the eval board — prove it works
- Open existing reference designs
- Plan firmware update mechanism (DFU)
- Reserve a debug header on PCB

Key Takeaways

Key takeaways from this lecture

32-bit by default

Cortex-M cheaper than 8-bit at moderate volume.

Peripherals decide

Count required peripherals first.

Sleep current = life

Battery products live or die here.

Toolchain matters

Free SDK, free debugger, big community.

Buy the eval board

Test the part before designing it in.

Lifecycle awareness

NRND parts are not for new designs.

Recommended Reading

MCU resources

- Joseph Yiu — 'Definitive Guide to ARM Cortex-M' (Cortex-M0/M3/M4/M7 series)
- Miro Samek — Modern Embedded Systems Programming (YouTube)
- STM32CubeIDE / MCUXpresso / Nordic nRF Connect SDK — free vendor IDEs
- Embedded Artistry — open-source embedded resources
- PlatformIO — cross-vendor toolchain
- Mbed-OS / Zephyr / FreeRTOS docs
- DigiKey / Mouser parametric search for MCU selection
- EEVblog forum — strong embedded discussion community

Questions & Discussion

Ask anything — concepts, projects, sourcing, career.

References & Further Learning

- Joseph Yiu — Definitive Guide to ARM Cortex-M
- Miro Samek — Modern Embedded Systems Programming (YouTube)
- Vendor SDKs: STM32Cube, NXP MCUXpresso, Nordic nRF Connect
- PlatformIO — cross-vendor toolchain (free)
- Zephyr / FreeRTOS — open-source RTOS
- DigiKey / Mouser parametric search
- Reference designs at hackster.io, hackaday.io
- Local startup ecosystem (Bengaluru / Hyderabad) — talk to designers

Thank you • Build something this week.